

第 14 节 实操任务 3：配置每日推送

学习目标：daily_digest.py 手动运行成功 + Cron 定时任务已配置
前置要求：实操 1-2 已完成

背景

手动运行不是产品。产品需要无人值守、按时推送。本实操把 formatter + publish 串联成定时任务。

以下代码可以用 **OpenCode**、**Claude Code**、**Cursor**、**Trae** 或**通义灵码**等任意 AI 编程工具生成。

步骤 1：用 AI 编程工具生成 daily_digest.py

提示词：

```
1 请帮我编写 ~/ai-knowledge-base/v4-production/daily_digest.py 每日推送入口脚本
2
3 需求：
4 1. 从 formatter.py 加载文章
5 2. 过滤 relevance_score < 60 的低质量文章
6 3. 调用 publisher.py 的 publish_daily_digest() 推送到所有渠道
7 4. 打印推送结果汇总（成功/失败渠道数）
8 5. 没有高质量文章时跳过推送并打印警告
9
10 编码规范：PEP 8, 包含 if __name__ == "__main__" 入口
```

生成的代码：（参考实现）

```
1 """daily_digest.py - 每日推送入口。"""
2
3 import asyncio
4 import logging
5 from distribution.publisher import publish_daily_digest
6
7
8 async def main():
9     logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(level
10
11     print("=" * 50)
12     print("  AI 知识库 - 每日摘要推送")
13     print("=" * 50)
14
```

```

15     # 调用统一发布入口：生成简报 + 并发推送到所有渠道
16     results = await publish_daily_digest()
17
18     # 汇总
19     success = sum(1 for r in results if r.success)
20     print(f"\n推送结果: {success}/{len(results)} 个渠道成功")
21     for r in results:
22         status = "✅" if r.success else "❌"
23         print(f"    {status} {r.channel}: {r.message_id or r.error}")
24
25
26 if __name__ == "__main__":
27     asyncio.run(main())

```

步骤 2：手动运行测试

```

1 cd ~/ai-knowledge-base/v4-production
2 python3 daily_digest.py

```

期望输出：

```

1  =====
2      AI 知识库 - 每日摘要推送
3  =====
4
5  [INFO] [发布] 完成：1/2 个渠道成功
6
7  推送结果：1/2 个渠道成功
8      ✅ telegram: 12345678
9      ❌ feishu: 未配置 FEISHU_WEBHOOK_URL

```

步骤 3：创建 OpenClaw daily-digest Skill

OpenClaw Cron 需要通过 Skill 触发，所以先创建一个 Skill 描述文件。

```

1 mkdir -p ~/ai-knowledge-base/v4-production/openclaw/skills/daily-digest

```

参考实现：`v4-production/openclaw/skills/daily-digest/SKILL.md`

```

1  ---
2  name: daily-digest
3  description: 生成今日 AI 技术简报，汇总当天采集的 Top 5 知识条目，按相关性排序
4  allowed-tools:
5      - Read
6  ---
7
8  # 每日简报技能

```

```

9
10 ## 触发条件
11
12 当用户想要查看今日 / 本周 AI 技术汇总时激活。
13 典型触发词：简报、摘要、今日、daily、digest、briefing
14
15 ## 生成流程
16
17 > **重要 · 只允许 Read**：本技能不能用 Glob / Grep / exec。所有操作从 `Read kn
18
19 ### Step 1：读索引定位今日数据
20
21 用 `Read` 读 `knowledge/articles/index.json`（含每篇文章的 `id` / `title` /
22
23 **在内存里筛**今日的条目（`collected_at` 字段以今日日期开头）；今日无数据则回退到最
24
25 > **不要尝试 Glob 或 grep 文件名** — 索引文件已经聚合了所有元信息，一次 Read 就
26
27 ### Step 2：内存过滤 + 排序
28
29 1. 过滤 `relevance_score >= 0.6` 的条目
30 2. 按 `relevance_score` 降序排序
31 3. 取 Top 5
32
33 只对最终要进简报的 Top 5，用 `Read knowledge/articles/{id}.json` 拿 `summary
34
35 ### Step 3：按 category 分组生成 Markdown 简报
36
37 ## 与 Publisher 的分工
38
39 - **本 Skill**：格式化文本，生成 Markdown 简报
40 - **distribution/publisher.py**：把简报推送到 Telegram / 飞书 / 文件
41
42 Skill 负责"写"，Publisher 负责"发"。

```

创建后立即重启 **daemon** 让 Bot 加载新 skill：`openclaw daemon restart`

步骤 4：配置 Cron 定时任务

主线方案：系统 Crontab

```
1 crontab -e
```

添加（注意用绝对路径 + python3 全路径）：

```

1 # 每天早上 8:00 推送
2 0 8 * * * cd /home/$USER/ai-knowledge-base/v4-production && /usr/bin/pyt

```

确保日志目录存在：

```
1 mkdir -p ~/ai-knowledge-base/v4-production/logs
```

步骤 5 : 验证 Cron 配置

```
1 crontab -l | grep daily_digest
2 # 应看到刚加的那一行
```

快速验证（用 1 分钟后触发）

如果不想等到第二天 8 点。临时改成 1 分钟后跑一次：

```
1 NEXT=$(date -d "+1 minute" +"%M %H")
2 (crontab -l 2>/dev/null; echo "$NEXT * * * cd /home/$USER/ai-knowledge-b
3 sleep 80
4 cat ~/ai-knowledge-base/v4-production/logs/cron-test.log
```

看到正常输出 → cron 环境 OK。然后 `crontab -e` 删测试行恢复正式配置。

步骤 6 : 提交到 Git

```
1 git add daily_digest.py openclaw/skills/daily-digest/
2 git commit -m "feat: add daily digest pipeline + skill + cron schedule"
```

Cron 最佳实践

实践	说明
日志重定向	<code>>> logs/cron.log 2>&1</code>
绝对路径	Cron 没有 PATH
幂等性	重复运行不产生重复推送
超时控制	<code>timeout 300 python3 daily_digest.py</code>

完成！ 每日推送已自动化。第 14 节实操全部完成。